

Manim Mobject 速查手册

1. 数字

1.1. `DecimalNumber` 任意整数或浮点数

1.2. `Integer` 整数值

1.3. `Variable` 动态展示变量

2. Latex公式

2.1. `Title` 标题

2.2. `BulletedList` 列表

2.3. `SingleStringMathTex` 单行数学公式

2.4. `MathTex` 多行数学公式

2.5. `Tex` 数学公式

3. 文本

3.1. `Text` 纯文本渲染器

3.2. `MarkupText` 富文本渲染器

3.3. `Paragraph` 多行纯文本

3.4. `Code` 代码展示

4. 表格和矩阵

4.1. `Table` 表格

4.2. `Matrix` 矩阵

5. 几何体

5.1. `Dot` (含 `LabeledDot` / `AnnotationDot`)

5.2. 圆形类 (`Circle` / `Annulus` / `Ellipse`)

5.3. 圆弧形类 (`Arc` / `ArcBetweenPoints` / `AnnularSector` / `Sector`)

5.4. 直线类 (`Line` / `DashedLine` / `TangentLine` / `LabeledLine`)

5.5. 带箭头直线类 (`Arrow` / `DoubleArrow` / `LabeledArrow` / `Vector`)

5.6. 曲线类

5.7. 角度标记

5.8. 常用多边形

5.9. 通用多边形

5.10. 弧形多边形 (ArcPolygon / ArcPolygonFromArcs)

5.11. Cutout (空心多边形)

5.12. 图形间集合关系

5.13. 形状匹配

6. 图结构

6.1. 无向图 Graph

6.2. 有向图 (DiGraph)

7. 3D结构

7.1. 三维的点与线

7.2. 立方体 Cube & 棱柱体 Prism

7.3. 圆锥 Cone

7.4. 球体 Sphere

7.5. 圆环面 Torus

7.6. 圆柱体 Cylinder

7.7. 曲面 Surface

7.8. 多面体 Polyhedron & 正多面体

8. 坐标系

8.1. 数轴 NumberLine

8.2. 直角平面 NumberPlane

8.3. 复数平面 ComplexPlane

8.4. 极坐标平面 PolarPlane

8.5. 通用二维坐标系 Axes

8.6. 通用三维坐标系 ThreeDAxes

1. 数字

1.1. DecimalNumber 任意整数或浮点数

`DecimalNumber` 是 Manim 中“全能数字显示器”，可渲染任意整数或浮点数，并自带格式、单位、符号、逗号等全套排版选项。

核心参数

名称	类型/取值	说明	默认值
<code>number</code>	<code>float</code>	要显示的数值	<code>0</code>
<code>num_decimal_places</code>	<code>int</code>	保留小数位（自动四舍五入）	<code>2</code>
<code>include_sign</code>	<code>bool</code>	是否强制显示 <code>+/-</code> 号	<code>False</code>
<code>group_with_commas</code>	<code>bool</code>	千位分隔符（1,234,567）	<code>False</code>
<code>show_ellipsis</code>	<code>bool</code>	被截断时尾部加 <code>...</code> 提示	<code>False</code>
<code>unit</code>	<code>str</code>	附加单位文本，如 <code>"km/s"</code>	<code>""</code>
<code>unit_buff_per_font_unit</code>	<code>float</code>	数值与单位的间距系数	<code>0.06</code>
<code>include_background_rectangle</code>	<code>bool</code>	是否添加背景矩形	<code>False</code>
通用样式参数	同所有 <code>Mobject</code>	<code>font_size</code> 、 <code>color</code> 、 <code>stroke_width</code> 等	—

常用方法

方法	功能	示例
<code>get_value()</code>	返回当前存储的精确浮点值	<code>v = mob.get_value()</code>
<code>set_value(new_num)</code>	立即更新显示内容	<code>mob.set_value(3.14159)</code>

<code>increment_value(delta)</code>	在现有值上累加	<code>mob.increment_value(0.1)</code>
继承通用动画	<code>scale</code> , <code>rotate</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> 等均可使用	

典型陷阱

1. 四舍五入仅视觉生效, `get_value()` 仍返回原始浮点值; 若需真正截断请自行处理后再 `set_value` 。
2. 千位逗号仅在 `group_with_commas=True` 且整数部分 ≥ 4 位时才出现。
3. `unit_buff_per_font_unit` 与 `font_size` 联动,
字号变化时隔距会等比放大缩小, 手动微调时容易失衡。
4. 背景矩形默认带 0.2 透明度;
若想完全遮盖需额外设置 `BackgroundRectangle(mob, color=..., opacity=1)` 。
5. 频繁 `set_value` 会触发重新排版 (尤其是带逗号或单位时),
大量更新请使用 `ChangingDecimal` 或缓存策略。

1.2. Integer 整数值

`Integer` 是 `DecimalNumber` 的整数特化版, 用来在画面中干净地显示整数值。

核心参数

(继承自 `DecimalNumber` , 但小数位固定为 0, 故大部分情况下不必显式设置)

参数	作用	默认值 / 备注
<code>number</code>	要显示的初始数值 (整数或浮点均可)	0
<code>font_size</code>	字体大小 (points)	48
<code>color</code>	字体颜色	<code>WHITE</code>

<code>num_decimal_places</code>	强制为 0，不可修改	—
---------------------------------	------------	---

其余 `DecimalNumber` 的格式、字体、对齐等参数同样可用，但一般保持默认即可。

常用方法

方法	功能	示例
<code>get_value()</code>	返回四舍五入后的整数	<code>int_val = mob.get_value()</code>
<code>set_value(new_int)</code>	立即更新显示为新的整数	<code>mob.set_value(42)</code>
<code>increment_value(delta)</code>	在当前值上加整数或浮点，再四舍五入显示	<code>mob.increment_value(3.9)</code> → 实际加 4
继承通用变换	<code>scale</code> , <code>set_color</code> , <code>set_x</code> , <code>set_y</code> , <code>move_to</code> , <code>rotate</code> , <code>fade_in/out</code> , <code>animate</code> 等均可直接使用	

典型陷阱

1. 四舍五入：任何浮点输入都会被先四舍五入再取整，`get_value()` 永远返回 `int` 类型。
2. 与 `DecimalNumber` 混用：若后续需要显示小数，应改用 `DecimalNumber` ；
反之亦然，避免中途切换造成闪烁。
3. 性能：频繁 `set_value` 会触发重排和重绘，对大量数字动画建议用 `ChangingDecimal` 或手动缓存。
4. 版本差异：Manim Community ≥ 0.15 起 `Integer` 才暴露 `num_decimal_places=0` 的强制行为，
老版本可能仍可手动设置小数位（不推荐）。

1.3. Variable 动态展示变量

`Variable` 把“标签 + 数值”打包成一块实时刷新的 Mobject,

用来在动画中**动态展示变量**（整数或浮点均可）。

核心参数

名称	类型/取值	作用	默认
<code>var</code>	<code>float</code>	初始数值	<code>0</code>
<code>label</code>	<code>str</code> / <code>Tex</code> / <code>MathTex</code> / <code>Text</code> ...	左侧标签（变量名）	<code>"x"</code>
<code>var_type</code>	<code>Integer</code> 或 <code>DecimalNumber</code>	决定数值显示方式	<code>DecimalNumber</code>
<code>num_decimal_places</code>	<code>int</code>	保留小数位（仅当 <code>var_type=DecimalNumber</code> 生效）	<code>2</code>

其余 `DecimalNumber` / `Integer` 的参数（颜色、字体大小、单位、逗号等）会透传给内部的数值对象。

常用方法

方法/属性	功能	示例
<code>tracker</code>	内置 <code>ValueTracker</code> ，用来实时读写数值	<code>var.tracker.get_value()</code>
<code>set_value(v)</code>	立即更新显示	<code>var.tracker.set_value(3.14)</code>
<code>increment_value(d)</code>	动画式递增	<code>self.play(var.tracker.animate.increment_value(1))</code>

<code>add_updater(fun c)</code>	让变量随别的量实时变化	<code>var.add_updater(lambda m: m.tracker.set_value (t.get_value()*2))</code>
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>align_to</code> , <code>animate</code> 等均可直接 调用	

典型陷阱

1. 四舍五入只在显示层生效: `var_type=Integer` 时, 即使内部 `tracker` 存的是 `3.9` , 画面仍显示 `4` , 但 `get_value()` 返回 `3.9` 。
2. `label` 接受任意 Mobject, 但**不要**传入可变对象 (如另一个 `Variable`) , 否则可能导致循环更新或渲染错误。
3. 大量 `add_updater` 时, 注意性能: 每个 `updater` 每帧都会执行, 复杂计算请用外部函数缓存。
4. 旧版本 (<0.15) 中 `Variable` 没有 `tracker` 属性, 需手动绑定 `ValueTracker` , 升级后脚本需同步调整。

2. Latex公式

2.1. Title 标题

`Title` 是一行**置顶标题** + **可选下划线**的快捷封装, 自动居中在画面顶端, 支持 LaTeX 与多段文字混合。

核心参数

名称	类型	作用	默认
<code>text_parts</code>	<code>`list[str]</code>	Tex	MathTex
<code>include_underline</code>	<code>bool</code>	是否绘制下划线	<code>True</code>

<code>match_underline_width_to_text</code>	<code>bool</code>	下划线长度是否跟随文字宽度	<code>False</code> (全屏宽)
<code>underline_buff</code>	<code>float</code>	文字与下划线垂直间距	<code>0.15</code>
通用外观参数	继承 <code>Tex</code> / <code>MathTex</code>	<code>color</code> 、 <code>font_size</code> 、 <code>stroke_width</code> 等	—

常用方法

方法	功能	示例
<code>set_text(*new_parts)</code>	运行时更新标题文字	<code>title.set_text("Step 2")</code>
继承通用动画	<code>scale</code> 、 <code>rotate</code> 、 <code>move_to</code> 、 <code>set_color</code> 、 <code>fade_in/out</code> 、 <code>animate</code> 等均可使用	

典型陷阱

1. 下划线宽度默认等于屏幕宽度；
若只想与文字等宽，务必手动设 `match_underline_width_to_text=True`。
2. `text_parts` 按顺序拼接，不会自动加空格，多段文字需自己留空格或用 `Text(" ")` 占位。
3. `Title` 在初始化时已将整体 `move_to(UP * 3.5)`；
若场景顶部有自定义边距，需 `title.shift(DOWN * 0.5)` 重新微调。
4. 旧版 Manim 中 `Title` 位于 `manim.scene.moving_camera_scene`，
导入路径与新版不一致，升级后脚本需检查 `from manim import Title`。

2.2. BulletedList 列表

BulletedList 是一键生成的垂直对齐的圆点列表，自带行距、圆点大小与“突出单行”动画，适合快速做要点展示。

核心参数

名称	类型	作用	默认值
items	list[str Tex MathTex Text]	列表内容，可混排公式	[]
buff	float	行间距（Mobject 单位）	0.3
dot_scale_factor	float	圆点缩放倍数	1.0
tex_environment	str	请勿设置，官方预留	—
通用外观参数	继承 VGroup	颜色、对齐、缩放、动画等	—

常用方法

名称	功能	示例
fade_all_but(index)	仅保留第 index 行高亮，其余半透明	blist.fade_all_but(2)
继承通用操作	scale, move_to, set_color, animate, fade_in/out 等均可直接使用	

典型陷阱

1. `tex_environment` 在内部已做特殊处理，手动指定会破坏排版；
如需矩阵或方程组请改用 `MathTex` + `align*` 自建。
2. `items` 中的字符串若含 `$` 会被自动解析为 LaTeX，双反斜杠需写成 `\\`。
3. 圆点大小随 `dot_scale_factor` 线性放大，但不会随 `buff` 自动调整，
过大时会出现圆点与文字重叠，需手动微调 `buff`。
4. `fade_all_but` 只是透明度动画，不会移动摄像机或缩放被高亮行；
若需聚焦可额外 `self.play(FocusOn(...))`。

2.3. SingleStringMathTex 单行数学公式

`SingleStringMathTex` 是 Manim 里最轻量的单行数学公式渲染器，把一条 LaTeX 字符串直接变成可动画的数学对象。

核心参数

名称	类型	作用	默认值
<code>tex_string</code>	<code>str</code>	LaTeX 公式源码（仅 单行）	<code>""</code>
<code>stroke_width</code>	<code>float</code>	轮廓线粗细	<code>2.0</code>
<code>height</code>	<code>float</code>	整体目标高度（缩放用）	<code>None</code> （按字体大小自适应）
<code>font_size</code>	<code>float</code>	字体大小（points）	<code>48</code>
<code>tex_environment</code>	<code>str</code>	LaTeX 环境名，非必要勿改	<code>"align*"</code>
<code>tex_template</code>	<code>TexTemplate</code>	自定义模板，高级用法	默认模板

常用方法

名称	功能	示例
<code>get_tex_string()</code>	返回最初传入的 LaTeX 源码	<code>print(mob.get_tex_string())</code>

<code>init_colors()</code>	预留钩子，子类可重写做自定义染色	目前空实现
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> , <code>rotate</code> e 等均可直接使用	

典型陷阱

1. 仅支持单行：换行符 `\\` 或 `align` 环境会被整体装进一行，导致排版异常；
多行公式请改用 `MathTex` 。
2. `height` 与 `font_size` 同时设置时，`height` 优先生效，
会把字体强制缩放到给定高度，可能出现失真。
3. `tex_environment` 改动后若出现编译错误，多半是环境不兼容；恢复默认即可。
4. 特殊宏（如 `\require{cancel}`）需在 `tex_template` 里额外引入包，否则编译失败；
不懂 LaTeX 时保持默认最安全。

2.4. MathTex 多行数学公式

`MathTex` 是 Manim 的主力多行/分段数学公式渲染器，可把复杂 LaTeX 拆成可独立控制的小片段，方便逐字符动画、着色及变换。

核心参数

名称	类型	作用	默认值
<code>tex_strings</code>	<code>list[str]</code>	若干 LaTeX 片段，最终顺序拼接	<code>[]</code>
<code>arg_separator</code>	<code>str</code>	片段之间的分隔符	<code>" "</code> （一个空格）
<code>substrings_to_isolate</code>	<code>list[str]</code>	需要独立成子 Mobject 的子串	<code>[]</code>
<code>tex_to_color_map</code>	<code>dict[str, color]</code>	一键给指定子串上色	<code>{}</code>

<code>tex_environment</code>	<code>str</code>	LaTeX 环境（如 <code>align*</code> 、 <code>equation*</code> ）	<code>"align*"</code>
通用外观参数	继承 <code>SingleStringMathTex</code>	<code>font_size</code> 、 <code>stroke_width</code> 、 <code>height</code> 等	—

常用方法

名称	功能	示例
<code>get_part_by_tex(substr)</code>	获取 第一次出现 该子串的片段	<code>eq.get_part_by_tex("x")</code>
<code>get_parts_by_tex(substr)</code>	获取 所有出现 该子串的片段	<code>eq.get_parts_by_tex("+")</code>
<code>index_of_part(part)</code>	已知子片段，求其在整体中的索引	<code>eq.index_of_part(sub)</code>
<code>index_of_part_by_tex(substr)</code>	已知子串，求其首次出现的索引	<code>eq.index_of_part_by_tex("=")</code>
<code>set_color_by_tex(substr, color)</code>	给指定子串上色	<code>eq.set_color_by_tex("x", RED)</code>
<code>set_color_by_tex_to_color_map(dict)</code>	批量上色	<code>eq.set_color_by_tex_to_color_map({"x": RED, "y": BLUE})</code>
<code>set_opacity_by_tex(substr, opacity)</code>	给指定子串设透明度	<code>eq.set_opacity_by_tex("x", 0.3)</code>
<code>sort_alphabetically()</code>	把内部子片段按 ASCII 排序	<code>eq.sort_alphabetically()</code>

继承通用动画	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> 等均可使用	
--------	---	--

典型陷阱

1. 子串隔离优先级: `substrings_to_isolate` 必须 按最长→最短 顺序排列, 否则短串先匹配会把长串截断。
2. `arg_separator=""` 时相邻片段会无缝黏在一起, 容易漏空格; 需要空格请显式写 `"~"` 或 `"\quad"` 。
3. `tex_environment="equation"` 等单行环境 不支持 `\\` 换行; 要换行请用 `align*` 或 `split` 。
4. 频繁调用 `sort_alphabetically()` 会打乱原有逻辑顺序, 若后面仍需按原顺序动画, 请先 `copy()` 备份。

2.5. Tex 数学公式

`Tex` 是 `MathTex` 的“文本友好版”, 可在同一句中混排普通文字与`LaTeX` 公式, 只需用 `$...$` 包裹公式即可。

核心参数

(与 `MathTex` 完全一致, 额外行为差异见下方)

名称	类型	作用	默认值
<code>tex_strings</code>	<code>list[str]</code>	待渲染的字符串序列	<code>[]</code>
<code>arg_separator</code>	<code>str</code>	片段之间分隔符	<code>" "</code>
<code>substrings_to_isolate</code>	<code>list[str]</code>	需拆分的子串	<code>[]</code>
<code>tex_to_color_map</code>	<code>dict</code>	一键配色字典	<code>{}</code>
<code>tex_environment</code>	<code>str</code>	<code>LaTeX</code> 环境	<code>"align"</code>

通用外观参数	继承 <code>MathTex</code>	<code>font_size</code> 、 <code>color</code> <code>r</code> 等	—
--------	-------------------------	---	---

常用方法

名称	功能	示例
继承全部 <code>MathTex</code> 方法	<code>get_part_by_tex</code> , <code>set_color_by_tex</code> , <code>animate</code> ...	同 <code>MathTex</code>
混合文本/公式	普通文本直接写，公式用 <code>\$...\$</code> 包裹	<code>Tex("令 $x^2+y^2=r^2$ 时")</code>

典型陷阱

1. 必须 `$...$` 包裹公式：若遗漏 `$`，整句会被当作文本，公式符号将原样输出。

```

1 Tex("x+y")      # 显示为普通文本 x+y
2 Tex("$x+y$")    # 正确渲染为公式

```
2. 切勿在 `MathTex` 里加 `$$`：`MathTex("$x+y$")` 会直接报错，而 `Tex` 必须加。
3. 文本与公式混排时，空格需显式写出：`Tex("文本$公式$文本")` 之间无自动空格，必要时插入 `\,` 或 `\quad`。
4. 当公式占比高时，`Tex` 的排版速度略慢于 `MathTex`；
 若整句都是公式，请优先 `MathTex` 以获得更简洁代码。

3. 文本

3.1. Text 纯文本渲染器

`Text` 是 Manim 里最纯粹的纯文本渲染器，支持系统字体、富文本样式（粗/斜/渐变/局部着色）及多行排版，零公式场景首选。

核心参数

名称	类型	作用	默认值
<code>text</code>	<code>str</code>	要显示的文本 (可含 <code>\n</code> 换行)	<code>""</code>
<code>font</code>	<code>str</code>	系统字体名	<code>"Consolas"</code> (跨平台后备)
<code>font_size</code>	<code>float</code>	字体大小 (points)	<code>24</code>
<code>warn_missing_fonts</code>	<code>bool</code>	字体缺失时是否弹出警告	<code>True</code>
<code>weight</code>	<code>int str</code>	字重 (BOLD/SEMIBOLD/ 100...900)	<code>NORMAL</code>
<code>slant</code>	<code>int str</code>	字形 (ITALIC/OBLIQUE)	<code>NORMAL</code>
<code>line_spacing</code>	<code>float</code>	多行时的行距倍数	<code>1.15</code>
<code>color</code>	<code>str Color</code>	整体颜色	<code>WHITE</code>
<code>gradient</code>	<code>tuple[Color]</code>	整体线性渐变色	<code>None</code>
<code>t2c</code>	<code>dict[str, Color]</code>	局部字符→颜色映射	<code>{}</code>
<code>t2f</code>	<code>dict[str, str]</code>	局部字符→字体映射	<code>{}</code>
<code>t2g</code>	<code>dict[str, tuple[Color]]</code>	局部字符→渐变映射	<code>{}</code>
<code>t2s</code>	<code>dict[str, int]</code>	局部字符→字形映射	<code>{}</code>
<code>t2w</code>	<code>dict[str, int]</code>	局部字符→字重映射	<code>{}</code>
<code>stroke_width</code>	<code>float</code>	描边粗细	<code>0</code>
<code>height / width</code>	<code>float</code>	整体目标高/宽 (会等比缩放)	<code>None</code>

常用方法

名称	功能	示例
<code>font_list()</code>	类方法，返回系统可用字体列表	<code>print(Text.font_list())</code>
<code>init_colors()</code>	占位钩子，子类可重写	当前空实现
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> 等均可直接使用	

典型陷阱

- 1. 字体跨平台差异：Windows 常用 `Consolas` 、 `SimHei` ； macOS 常用 `Menlo` 、 `PingFang SC` ；
脚本迁移时需检测 `font_list()` 。
- 2. `t2*` 字典的 key 只支持连续子串匹配且区分大小写；若出现重叠，后定义的覆盖先定义。
- 3. `height` / `width` 会强制整体缩放，可能导致文字畸变；更推荐用 `font_size` 控制字号。
- 4. 换行请用 `\n` ；手动设置 `line_spacing<1` 时可能出现行重叠，需实时预览。
- 5. 若文本中包含 `$` 会被原样输出，不会触发公式渲染；需要公式请改用 `Tex` 或 `MathTex` 。

3.2. MarkupText 富文本渲染器

`MarkupText` 是 Manim 的 富文本渲染器：在普通字符串里直接写 HTML / Pango-Markup，即可得到粗斜体、上下标、划线、渐变、局部换色等高级排版效果。

核心参数

名称	类型	作用	默认值
<code>text</code>	<code>str</code>	带 HTML 标签的文本	<code>""</code>
<code>font</code>	<code>str</code>	全局字体	<code>"Consolas"</code>

font_size	float	字号 (points)	24
color	Color	全局颜色	WHITE
line_spacing	float	多行行距倍数	1.15
justify	bool	两端对齐	False
should_center	bool	水平居中	True
gradient	tuple[Color]	整体线性渐变	None
stroke_width / fill_opacity	float	描边粗细 / 透明度	0 / 1
height / width h	float	强制整体高/宽	None

常用内嵌标签速查

标签	示例	效果
<code>text</code>	<code>粗体</code>	粗体
<code><i>text</i></code>	<code><i>斜体</i></code>	<i>斜体</i>
<code><sup>...</sup></code> / <code><sub>...</sub></code>	<code>x<sup>2</sup></code>	上标 / 下标
<code>... </code>	<code>...</code>	局部字体
<code>... </code>	<code>红 字 </code>	前景色
<code>... </code>	<code> 蓝底</code>	背景色

<pre><gradient from="A" to="B">... </gradient></pre>	<pre><gradient from="red" to="blue">渐变 </gradient></pre>	线性渐变
<pre>... </pre>	见示例	单下划线
<pre>... </pre>	见示例	单上划线
<pre>... </pre>	见示例	删除线
<pre>... </pre>	见示例	波浪下划线

常用方法

名称	功能	示例
<code>font_list()</code>	类方法：列出系统所有可用字体	<code>print(MarkupText.font_list())</code>

继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> 等均可直接使用	
--------	---	--

典型陷阱

- 1. 标签必须闭合；漏写 `</span>` 会导致整段渲染失败并报 LaTeX 错误。
- 2. 颜色/字体名需用英文小写或十六进制（如 `#ff0000` ），中文名可能解析失败。
- 3. `underline="error"` 的波浪线仅在 Pango ≥ 1.44 生效，低版本会退化为单下划线。
- 4. 使用 `<gradient>` 时，标签内不得再嵌套其他 `<span>` ，否则渐变会被截断。
- 5. `stroke_width` 对大多数字体有效，但对部分无轮廓字体（如点阵字体）无效果。

3.3. Paragraph 多行纯文本

`Paragraph` 是多行纯文本 的排版利器，一次性传入若干字符串即可生成等宽段落，支持行距、对齐、`t2*` 局部样式等全部 `Text` 级功能。

核心参数

名称	类型	作用	默认值
<code>text</code>	<code>list[str]</code>	多行内容，每元素一行	<code>[]</code>
<code>line_spacing</code>	<code>float</code>	行距倍数	<code>1.15</code>
<code>alignment</code>	<code>str</code>	每行对齐方式	<code>"left"</code>
继承 <code>Text</code> 全部参数	<code>font</code> , <code>font_size</code> , <code>color</code> , <code>t2c</code> , <code>t2f</code> , <code>t2g</code> , <code>t2w</code> , <code>t2s</code> , <code>stroke_width</code> , <code>height</code> , <code>width</code> ...	与 <code>Text</code> 完全一致	同 <code>Text</code>

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> 等均可直接使用	—

典型陷阱

- 1. `text` 必须是可迭代字符串；若误传单个长字符串，会被拆成逐字符行，导致排版崩坏。
- 2. `alignment="center"` 仅对各行生效，不会整体居中；
如需整体居中请再 `paragraph.move_to(ORIGIN)` 。
- 3. `t2c` / `t2g` 等字典的 key 仅在单行内部匹配；跨行子串需每行分别定义。
- 4. `height` / `width` 强制缩放会改变行距比例；
建议在 `scale` 后再微调 `line_spacing` 以保持阅读舒适。

3.4. Code 代码展示

`Code` 是 Manim 的代码展示专用 `Mobject`，可自动语法高亮、加行号、换行距、加背景，并支持 40+ 主题样式，一行代码即可把整段程序搬进动画。

核心参数

名称	类型	说明	默认/常用值
<code>file_name</code>	<code>str</code>	从文件读取代码（与 <code>code</code> 二选一）	<code>None</code>
<code>code</code>	<code>str</code>	直接传入代码字符串	<code>""</code>
<code>language</code>	<code>str</code>	语法高亮语言（ <code>python</code> / <code>java</code> / <code>c</code> / <code>go</code> ...）	<code>"python"</code>

<code>style</code>	<code>str</code>	高亮主题 (<code>monokai</code> 、 <code>github-dark</code> 、 <code>vim</code> ...)	<code>"default"</code>
<code>tab_width</code>	<code>int</code>	Tab 转空格数	<code>4</code>
<code>line_spacing</code>	<code>float</code>	行距倍数	<code>1.15</code>
<code>font_size</code>	<code>float</code>	字号	<code>24</code>
<code>font</code>	<code>str</code>	系统字体名	<code>"Consolas"</code>
<code>margin</code>	<code>float</code>	文本到背景内边距	<code>0.3</code>
<code>insert_line_no</code>	<code>bool</code>	是否显示行号	<code>True</code>
<code>line_no_from</code>	<code>int</code>	起始行号	<code>1</code>
<code>line_no_buff</code>	<code>float</code>	行号与代码间距	<code>0.4</code>
<code>background</code>	<code>"rectangle" "window"</code>	背景形状	<code>"rectangle"</code>
<code>background_stroke_width</code>	<code>float</code>	背景边框粗细	<code>2</code>
<code>background_stroke_color</code>	<code>Color</code>	背景边框颜色	<code>WHITE</code>
<code>corner_radius</code>	<code>float</code>	背景圆角半径	<code>0.2</code>
<code>generate_html_file</code>	<code>bool</code>	同时导出高亮 HTML 文件	<code>False</code>
<code>warn_missing_font</code>	<code>bool</code>	字体缺失警告	<code>True</code>

主题列表: `print(Code.styles_list)` → 40+ 内置主题任选。

常用方法

名称	功能	示例
<code>styles_list</code>	类属性: 列出所有可用主题	<code>print(Code.styles_list)</code>

继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> 等均可直接使用	—
--------	--	---

典型陷阱

1. `file_name` 与 `code` 只能二选一，同时给出以 `file_name` 为准。
2. `language` 必须小写；写成 `"Python"` 会高亮失败并退化为无高亮文本。
3. `background="window"` 时，若代码较短，窗口会留空；可配合 `margin` 调整。
4. 行号宽度随行数自动扩展，大文件（>1000 行）建议关闭 `insert_line_no`，否则左侧过宽。
5. 导出 HTML 文件固定目录：`assets/codes/generated_html_files`，清理项目时别误删。

4. 表格和矩阵

4.1. Table 表格

`Table` 家族是 Manim 的 **二维数据排版引擎**：把任意 `Mobject`（数字、公式、图形）放进行列，自动生成边框、标签、背景，并提供行列单元格级动画接口。

核心参数

名称	类型	说明	默认
<code>table</code>	<code>list[list[Any]]</code>	二维数据，单元格可为字符串、 <code>Mobject</code> 等	<code>[]</code>
<code>row_labels</code>	<code>list[Mobject]</code>	行标题（最左列）	<code>None</code>
<code>col_labels</code>	<code>list[Mobject]</code>	列标题（最上行）	<code>None</code>
<code>top_left_entry</code>	<code>Mobject</code>	左上角内容（行列标题交叉处）	<code>None</code>
<code>v_buff</code> / <code>h_buff</code>	<code>float</code>	单元格垂直 / 水平间距	<code>0.4</code> / <code>0.6</code>

<code>include_outer_lines</code>	<code>bool</code>	是否绘制外边框	<code>False</code>
<code>line_config</code>	<code>dict</code>	线型、颜色、粗细	<code>{"stroke_width":2, "color":WHITE}</code>
<code>add_background_rectangles_to_entries</code>	<code>bool</code>	每个单元格加背景	<code>False</code>
<code>include_background_rectangle</code>	<code>bool</code>	整张表加背景	<code>False</code>
<code>background_rectangle_color</code>	<code>Color</code>	背景色	<code>BLACK</code>
<code>arrange_in_grid_config</code>	<code>dict</code>	对齐方式（如 <code>{"cell_alignment": LEFT}</code> ）	居中对齐

常用方法

名称	功能	示例
<code>get_rows()</code> / <code>get_columns()</code>	返回行 / 列 VGroup	<code>rows = table.get_rows()</code>
<code>get_cell(pos)</code>	按 (row, col) 取单元格	<code>cell = table.get_cell((2,3))</code>
<code>add_highlighted_cell(positions, color)</code>	高亮并返回该单元格	<code>table.add_highlighted_cell((1,1), RED)</code>

<code>set_row_colors(*colors)</code> / <code>set_column_colors(*colors)</code>	批量着色行列	<code>table.set_row_colors(BLUE, GREEN)</code>
<code>get_horizontal_lines()</code> / <code>get_vertical_lines()</code>	获取横 / 纵线对象	<code>table.get_horizontal_lines()[0].set_color(RED)</code>
<code>add_background_to_entries()</code>	给所有单元格加背景	<code>table.add_background_to_entries()</code>
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

专用快捷子类

类	说明	等价写法
<code>DecimalTable</code>	表格元素为 <code>DecimalNumber</code>	<code>Table(..., element_to_mobject=DecimalNumber)</code>
<code>IntegerTable</code>	表格元素为 <code>Integer</code>	<code>Table(..., element_to_mobject=Integer)</code>
<code>MathTable</code>	表格元素为 <code>MathTex</code>	<code>Table(..., element_to_mobject=MathTex)</code>
<code>MobjectTable</code>	表格元素为任意 <code>Mobject</code>	<code>Table(..., element_to_mobject=lambda x: x)</code>

典型陷阱

- 行列索引从 1 开始：`get_cell((1,1))` 是左上角数据单元格（非标签）。
- 传入字符串时，内部自动转 `MathTex`；若要纯文本，请显式用 `Text("内容")` 作为单元格。
- 设置 `row_colors` / `column_colors` 时，颜色数量不足会循环复用，超量则截断。

4. 大表格 (>20×20) 渲染开销大, 建议先用 `MobjectTable` 占位, 再按需替换为轻量级元素。

4.2. Matrix 矩阵

`Matrix` 家族把任意二维数据 (数字、公式、图形) 封装成带括弧的**标准矩阵**, 可逐行逐列、逐元素动画, 是线性代数演示的默认选择。

核心参数

名称	类型	作用	默认
<code>matrix</code>	<code>list[list]</code>	二维数组, 元素可为数字 / 字符串 / Mobject	<code>[[0]]</code>
<code>v_buff</code> / <code>h_buff</code> <code>f</code>	<code>float</code>	元素垂直 / 水平间距	<code>0.8</code> / <code>0.6</code>
<code>bracket_h_buff</code> / <code>bracket_v_buff</code>	<code>float</code>	括弧与元素水平 / 垂直间距	<code>0.1</code> / <code>0.25</code>
<code>left_bracket</code> / <code>right_bracket</code>	<code>str</code>	左右括弧: <code>"["</code> / <code>"("</code> / <code>"{"</code> / <code>""</code>	<code>"</code>
<code>stretch_bracket</code> <code>s</code>	<code>bool</code>	是否自动拉宽括弧	<code>True</code>
<code>bracket_config</code>	<code>dict</code>	括弧线型、颜色、粗细	<code>{}</code>
<code>element_to_mobject</code>	<code>Callable</code>	把元素转为 Mobject 的函数	<code>MathTex</code>
<code>element_to_mobject_config</code>	<code>dict</code>	传给上函数的额外配置	<code>{}</code>
<code>element_alignment_corner</code>	<code>np.ndarray</code>	元素对齐锚点	<code>ORIGIN</code>
<code>add_background_rectangles_to_entries</code>	<code>bool</code>	给每个元素加背景	<code>False</code>

<code>include_background_rectangle</code>	<code>bool</code>	给整个矩阵加背景	<code>False</code>
---	-------------------	----------	--------------------

快捷子类

类	元素类型	等价写法
<code>IntegerMatrix</code>	整数	<code>Matrix(..., element_to_mobject=Integer)</code>
<code>DecimalMatrix</code>	浮点	<code>Matrix(..., element_to_mobject=DecimalNumber)</code>
<code>MobjectMatrix</code>	任意 Mobject	<code>Matrix(..., element_to_mobject=lambda x: x)</code>

常用方法

名称	功能	示例
<code>get_rows()</code> / <code>get_columns()</code>	获取行 / 列 VGroup	<code>rows = m.get_rows()</code>
<code>get_entries()</code>	获取所有元素 VGroup	<code>entries = m.get_entries()</code>
<code>get_brackets()</code>	获取左右括号 VGroup	<code>brackets = m.get_brackets()</code>
<code>set_row_colors(*colors)</code> / <code>set_column_colors(*colors)</code>	批量染色行列	<code>m.set_row_colors(RED, BLUE)</code>
<code>add_background_to_entries()</code>	统一给元素加背景	<code>m.add_background_to_entries()</code>

继承通用操作	<code>scale</code> , <code>move_to</code> , <code>animate</code> , <code>fade_in/</code> <code>out</code> ...	—
--------	---	---

典型陷阱

1. 行列索引从 0 开始: `get_entries()[0]` 是第一行第一列元素。
2. 字符串元素默认转 `MathTex` , 若想保留纯文本请显式传入 `Text("内容")` 。
3. 大矩阵 (>10×10) 渲染慢, 可先 `MobjectMatrix` 占位, 再按需替换为轻量元素。
4. 括弧类型误写 (如 `"{"` 而非 `"\{"`) 会导致 LaTeX 编译失败。

5. 几何体

5.1. Dot (含 LabeledDot / AnnotationDot)

`Dot` 系列是 Manim 的“像素级”几何标记: 从简单圆点到带标签、带双色描边的注释点, 一键放缩、改色、加文字。

核心参数

类	通用参数	额外专属参数	默认值
<code>Dot</code>	<code>point</code> , <code>radius</code> , <code>color</code> , <code>fill_opacity</code> , <code>stroke_width</code>	—	<code>radius=0.08</code>
<code>LabeledDot</code>	继承 <code>Dot</code> 全部	<code>label</code> (str / Mobject)	<code>label=""</code>
<code>AnnotationDot</code>	继承 <code>Dot</code> 全部	<code>stroke_color</code> , <code>fill_color</code>	<code>stroke_color=WHITE</code> , <code>fill_color=BLACK</code>

`point` 为 3D 坐标; `radius` 以场景单位计。

常用方法

名称	功能	示例
继承通用操作	<code>move_to</code> , <code>scale</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. 半径单位是“Manim 单位”而非像素；`radius=1` 在默认摄像机会占满半屏。
- 2. `LabeledDot` 的 `label` 直接给中文字符串会报错，需包一层 `Text("中文")` ；
公式可直接给 LaTeX 字符串。
- 3. `AnnotationDot` 的 `stroke_width` 过大时圆点会被描边撑成“环形”；需同步调小 `radius` 。
- 4. 对大量点 (>1k) 动画时，请用 `VGroup` 合并或改用 `DotCloud` 以提升性能。

5.2. 圆形类 (Circle / Annulus / Ellipse)

圆形三兄弟把“圆、环、椭”一网打尽：
`Circle` 画正圆，`Annulus` 挖圆环，`Ellipse` 拉椭圆，一键调半径、粗细、颜色，还能三点成圆、环绕其它图形。

核心参数

类	专属参数	通用参数 (color / stroke_width / fill_opacity)	默认值
Circle	<code>radius</code>	同所有 Mobject	<code>radius=1</code>
Annulus	<code>inner_radius</code> , <code>outer_radius</code>	同上	<code>inner=1</code> , <code>outer=2</code>

Ellipse	<code>width</code> , <code>height</code>	同上	<code>width=2</code> , <code>height=1</code>
---------	--	----	--

常用方法

名称	功能	示例
<code>from_three_points(p1,p2,p3)</code>	三点唯一圆	<code>Circle.from_three_points(a,b,c)</code>
<code>point_at_angle(theta)</code>	取圆上角度点	<code>c.point_at_angle(PI/3)</code>
<code>surround(mob, buffer_factor=1.2)</code>	让圆环绕目标	<code>Circle().surround(square)</code>
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

1. 单位半径在默认摄像机下≈屏幕高的一半； `radius=0.5` 以内才是“点”级大小。
2. `Annulus` 的 `inner_radius` $\geq$ `outer_radius` 时会渲染成实心圆；需保持 `inner < outer`。
3. `Ellipse(width, height)` 若 `width == height` 自动退化为正圆，可用来“偷懒”画圆。
4. `surround` 的 `buffer_factor<1` 时圆形会嵌入目标内部，视觉上可能被遮挡。

5.3. 圆弧形类 (Arc / ArcBetweenPoints / AnnularSector / Sector)

圆弧形四件套把“圆弧、两点弧、环扇、扇形”一次打包：
用半径-角度、起点-终点或内外半径即可画出任意弧段，并可随时移动圆心、改色、加动画。

核心参数

类	关键专属参数	通用参数 (color / stroke_width / fill_opacity)
Arc	radius , start_angle , angle , arc_center	同所有 Mobject
ArcBetweenPoints	start , end , angle , radius	同上
AnnularSector	inner_radius , outer_radius , start_angle , angle	同上
Sector	inner_radius , outer_radius	同上 (固定 1/4 环)

角度单位为 弧度，0 弧对应圆心右侧；逆时针为正。

常用方法

名称	功能	示例
get_arc_center()	取当前圆心坐标	c = arc.get_arc_center() ()
move_arc_center_to(point)	移动圆心到新位置	arc.move_arc_center_to(ORIGIN)
继承通用操作	scale , move_to , rotate , animate , fade_in/out ...	—

典型陷阱

- 1. ArcBetweenPoints 的 angle 与 radius 不可同时生效；同时给出时仅 radius 被采用。
- 2. AnnularSector 必须满足 inner_radius < outer_radius ；相等或倒置会渲染成实心圆或空对象。

3. `Sector` 固定 90° (1/4 环) , 如需其它角度请改用 `AnnularSector` 。
4. 大半径 + 大角度会生成极长路径, 若出现卡顿可适当降低 `num_components` (默认 9, ≥ 15 已足够平滑) 。

5.4. 直线类 (Line / DashedLine / TangentLine / LabeledLine)

直线四件套：

`Line` 画任意线段, `DashedLine` 画虚线, `TangentLine` 自动生成切线, `LabeledLine` 在线上加标签；全部支持弯曲、改色、实时重定位。

核心参数

类	专属参数	通用参数 (start / end / color / stroke_width / fill_opacity)
Line	<code>path_arc</code> (弯曲弧度)	起点/终点必填
DashedLine	<code>dash_length</code> , <code>dashed_ratio</code>	继承 Line
TangentLine	<code>vmob</code> , <code>alpha</code> , <code>length</code> , <code>d_alpha</code>	—
LabeledLine	<code>label</code> , <code>label_position</code> , <code>font_size</code> , <code>label_color</code> , <code>label_frame</code> , <code>frame_fill_color</code> , <code>frame_fill_opacity</code>	继承 Line

常用方法

名称	功能	示例
<code>get_angle()</code>	获取与水平轴夹角	<code>ang = l.get_angle()</code>
<code>set_angle(theta)</code>	旋转到指定角度	<code>l.set_angle(PI/4)</code>
<code>set_length(len)</code>	改变长度 (中点不变)	<code>l.set_length(3)</code>

<code>put_start_and_end_on(p1,p2)</code>	重设两端	<code>l.put_start_and_end_on(a,b)</code>
<code>get_projection(point)</code>	点到直线投影	<code>proj = l.get_projection(P)</code>
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

1. `path_arc` 为弧度值，过大 ($>\pi$) 会出现自交；曲线动画建议分段。
2. `TangentLine` 的 `alpha` 取值 0~1，对应沿曲线一周；0.5 恰为对侧。
3. `LabeledLine` 的 `label` 若为中文，请用 `Text("中文")` 而非裸字符串，否则渲染报错。
4. 动态改变虚线 `dash_length` 或 `dashed_ratio` 后需重新 `self.play(Create(...))` 才能刷新显示。

5.5. 带箭头直线类 (`Arrow` / `DoubleArrow` / `LabeledArrow` / `Vector`)

箭头四件套：

`Arrow` 画单箭头直线，`DoubleArrow` 画双箭头，`LabeledArrow` 加标签，`Vector` 默认从原点出发——一条指令即可生成矢量、标注或流程图箭头。

核心参数

类	专属参数	通用参数 (start/end 或 direction / color / stroke_width ...)
---	------	---

Arrow	<code>buff</code> , <code>max_tip_length_to_length_ratio</code> , <code>max_stroke_width_to_length_ratio</code> , <code>tip_shape</code>	<code>start</code> , <code>end</code>
DoubleArrow	同上（双箭头）	同上
Vector	无额外独有参数	<code>direction</code> (终点, 起点固定原点)
LabeledArrow	<code>label</code> , <code>label_position</code> , <code>font_size</code> , <code>label_color</code> , <code>label_frame</code> , <code>frame_fill_color</code> , <code>frame_fill_opacity</code>	继承 Arrow

常用 `tip_shape`

`ArrowTriangleTip` , `ArrowTriangleFilledTip` , `ArrowSquareTip` , `ArrowSquareFilledTip` ,
`ArrowCircleTip` , `ArrowCircleFilledTip` , `StealthTip` ...

常用方法

名称	功能	示例
<code>get_default_tip_length()</code>	当前箭头头部长度	<code>l = arrow.get_default_tip_length()</code>
<code>coordinate_label(color=WHITE)</code>	Vector 专属: 自动生成坐标文本	<code>label = vec.coordinate_label(RED)</code>

继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—
--------	---	---

典型陷阱

1. `Vector` 只认 `direction`（相对原点），若需自定义起点请改用 `Arrow`。
2. `buff` 是线端到箭头根部的留白；设 0 时箭头会顶住终点，视觉效果挤。
3. `max_tip_length_to_length_ratio` 越大，箭头显得越“大”；过大会导致短线无法显示箭头。
4. `LabeledArrow` 的 `label` 如需中文或公式，
 请用 `Text("中文")` / `MathTex("x^2")`，裸字符串会报错。

5.6. 曲线类

曲线三件套：

`CubicBezier` 用 4 个控制点绘制任意三次贝塞尔曲线；
`CurvedArrow` / `CurvedDoubleArrow` 在两点间自动生成带单 / 双箭头的弧线，无需手动计算弧度。

核心参数

类	专属参数	通用参数（color / stroke_width / fill_opacity ...）
<code>CubicBezier</code>	<code>start_anchor</code> , <code>start_handle</code> , <code>end_handle</code> , <code>end_anchor</code> （4 个控制点）	—

CurvedArrow	<code>start</code> , <code>end</code> (继承 ArcBetweenPoints)	同 Arc
CurvedDoubleArrow	同上 (双箭头)	同上

4 点顺序对应贝塞尔公式：起点锚点 → 起点控制 → 终点控制 → 终点锚点。

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. CurvedArrow 的弧线方向固定为逆时针；如需顺时针，请交换起点/终点或使用负角度。
- 2. 当起点与终点距离过近且 angle 过大时，弧线会自交；适当减小 angle 或增大 radius 。
- 3. CubicBezier 的 4 点需成对出现，缺失任一控制点将退化为直线。
- 4. 绘制复杂图形（如爱心）时，建议先在纸上描点再转坐标，以免控制点过远导致曲线“飞出去”。

5.7. 角度标记

角度三件套：

Angle 在两条线间自动生成弧标； RightAngle 绘制直角拐角； Elbow 任意位置直角符号——均可调半径、位置、颜色，还能实时取值。

核心参数

类	专属参数	通用参数 (color / stroke_width / fill_opacity ...)
---	------	--

Angle	<code>line1</code> , <code>line2</code> , <code>radius</code> , <code>quadrant</code> , <code>other_angle</code> , <code>dot</code> , <code>dot_radius</code> , <code>dot_distance</code> , <code>dot_color</code> , <code>elbow</code>	同 Mobject
RightAngle	<code>length</code> （拐角大小）	继承 Angle
Elbow	<code>width</code> , <code>angle</code> （旋转角）	同 Mobject

常用方法

名称	功能	示例
<code>from_three_points(A,B,C)</code>	三点快速生成角标	<code>ang = Angle.from_three_points(A,B,C)</code>
<code>get_lines()</code>	取出构成角的两条线	<code>lines = ang.get_lines()</code>
<code>get_value(degrees=True)</code>	实时获取角度值	<code>print(ang.get_value())</code>
<code>get_arc_center()</code> (Angle)	取弧圆心	<code>c = ang.get_arc_center()</code>
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. `quadrant=(1,1)` 等 4 个组合决定弧落在哪个象限；写错会导致 270° 弧翻边。
- 2. `RightAngle` 只能作用于垂直的线，否则拐角会错位；任意角度请改用 `Elbow`。
- 3. `dot_distance` 是相对弧半径的倍数，`dot_distance=1` 时圆点恰落在弧上。
- 4. 大量角度动画时，先 `VGroup` 合并再整体 `animate` 可减少卡顿。

5.8. 常用多边形

多边形五件套：

`Triangle` 画等边三角形，`Square` 画正方形，`Rectangle` 画可切分的矩形，`RoundedRectangle` 画圆角矩形，`Star` 画星形——全部只需一行参数即可生成、缩放、旋转、着色。

核心参数

类	专属参数	通用参数 (color / stroke_width / fill_opacity ...)
Triangle	—	全部继承
Square	side_length	同上
Rectangle	width , height , grid_xstep , grid_ystep (自动切分网格)	同上
RoundedRectangle	corner_radius (可为单值或 [r1,r2,r3,r4])	继承 Rectangle
Star	n (尖角数) , outer_radius , inner_radius , density , start_angle	同上

常用方法

名称	功能	示例
继承通用操作	scale , rotate , move_to , animate , fade_in/out ...	—

典型陷阱

- `Rectangle` 的 `grid_xstep` / `grid_ystep` 必须能整除 `width` / `height` , 否则最后一格会被拉伸。

- 2. `RoundedRectangle` 若给 4 个角半径，顺序为左上→右上→右下→左下；不足 4 值会循环填充。
- 3. `Star` 的 `density` 仅在手动设置 `inner_radius` 后生效，默认 `density=2` 即可满足大部分需求。
- 4. 所有多边形默认锚点为中心；
若需左下角对齐，请配合 `move_to(LEFT*width/2+DOWN*height/2)` 手动调整。

5.9. 通用多边形

用顶点即可“徒手”画任意多边形：

`Polygon` 任意形状闭合，`RegularPolygon` n 边正形，`Polygram` 可画多组/开口，`RegularPolygram` 星形/跳点连——一参一点，一参 n 边。

核心参数

类	专属参数	通用参数 (color / stroke_width / fill_opacity ...)
<code>Polygon</code>	<code>*vertices</code> (可变参数：按顺序顶点)	全部继承
<code>RegularPolygon</code>	<code>n</code> (边数)	同上
<code>Polygram</code>	<code>*vertex_groups</code> (多组顶点，可闭合/开放)	同上
<code>RegularPolygram</code>	<code>num_vertices</code> , <code>radius</code> , <code>density</code> , <code>start_angle</code>	同上

常用方法

名称	功能	示例
----	----	----

<code>get_vertices()</code> / <code>get_vertex_groups()</code>	取顶点列表或分组	<code>print(pg.get_vertices())</code>
<code>round_corners(radius)</code>	一键倒角	<code>poly.round_corners(0.1)</code>
继承通用操作	<code>scale</code> , <code>rotate</code> , <code>move_to</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

1. `Polygon` 顶点顺序决定内外轮廓，逆时针为正方向，否则填充可能“镂空”。
2. `RegularPolygram` 的 `density` 为“跳点”步长；
`density=1` 即普通正多边形，`density>1` 生成星形。
3. `Polygram` 若只给一组顶点，行为与 `Polygon` 相同；多组顶点时各组独立，不自动闭合。
4. `round_corners` 倒角半径过大会把锐角“吃掉”，需小于最小边长一半。

5.10. 弧形多边形（ArcPolygon / ArcPolygonFromArcs）

用圆弧代替直线边，一键生成“圆角”多边形：

`ArcPolygon` 按顶点+弧度/半径批量生成；

`ArcPolygonFromArcs` 直接拼装现成 `Arc` ；

不闭合时自动补直线封口。

核心参数

类	专属参数	通用参数（color / stroke_width / fill_opacity ...）
<code>ArcPolygon</code>	<code>*vertices</code> （顶点列表） <code>angle</code> 或 <code>radius</code> （统一弧度） <code>arc_config</code> （每边单独 dict）	同 Mobject

ArcPolygonFromArcs	*arcs (Arc 对象序列)	同上
--------------------	------------------	----

`angle` 与 `radius` 二选一；同时给出时 `angle` 被忽略。

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. 顶点顺序决定内外轮廓，逆时针为正；反向会导致填充“镂空”。
- 2. `radius` 越大弧越“平”，`angle` 越大弧越“翘”；极端值会让边自交。
- 3. `ArcPolygonFromArcs` 若弧序列未闭合，系统会用直线自动封口，颜色/线型沿用默认直线设置，无法单独配置。

5.11. Cutout（空心多边形）

`Cutout` 是“布尔差”工具：给定一个主体形状 & 若干挖洞形状，自动在主体上挖出空洞，常用于面积演示、PCB 镂空或几何证明。

核心参数

名称	类型	说明	默认值
<code>main_shape</code>	VMobject	被挖的主体（外轮廓）	必填
<code>*mobjects</code>	VMobject...	任意数量的挖洞形状	必填
继承通用参数	color / stroke_width / fill_opacity ...	同所有 Mobject	—

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. 挖洞形状**必须完全落在主体内部**，否则会出现“反向镂空”或渲染异常。
- 2. 多个挖洞形状之间可以重叠；重叠区域仍视为一次挖空。
- 3. 动画中频繁改变 `fill_opacity` 时，内部空洞会随主体一起透明；
如需单独控制，请先把 `Cutout` 转成 `VObject` 。
- 4. 大量挖洞（>50）时建议先缓存为 PNG 或使用 `VGroup` 合并，避免每帧布尔计算带来的性能开销。

5.12. 图形间集合关系

布尔运算四件套：

`Difference` 差集 ($A-B$) , `Exclusion` 异或 ($A\oplus B$) , `Intersection` 交集 ($A\cap B$) , `Union` 并集 ($A\cup B$) ——一行代码完成任意 `VObject` 的布尔运算。

核心参数

类	必需参数	说明
<code>Difference</code>	<code>subject</code> , <code>clip</code>	从主体中减去与裁切体相交部分
<code>Exclusion</code>	<code>subject</code> , <code>clip</code>	保留 A、B 非交集部分
<code>Intersection</code>	<code>*vmobjects</code> (≥ 2)	保留所有形状的共同区域
<code>Union</code>	<code>*vmobjects</code> (≥ 2)	合并所有形状

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>set_color</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. 运算结果依赖 **顶点方向**（顺时针/逆时针）；方向相反会导致填充镂空。
- 2. 大量布尔运算（>20 次）会显著拖慢渲染，建议先缓存为 PNG 或 `Mobject` 后复用。
- 3. `Intersection` / `Union` 至少传 2 个对象，否则会抛 `ValueError` 。
- 4. 动画中实时改变原始形状会导致布尔结果每帧重算，量大时请改用静态结果。

5.13. 形状匹配

“四件套标注”——给任意对象瞬间加背景、叉、框、下划线，用于突出、解释或强调，无需手动计算尺寸和位置。

核心参数

类	专属参数	通用参数（color / stroke_width / fill_opacity ...）
BackgroundRectangle	<code>mobject</code> , <code>buff</code> , <code>color</code> , <code>fill_opacity</code> , <code>stroke_width</code> , <code>stroke_opacity</code>	同 Mobject
Cross	<code>mobject</code> , <code>stroke_color</code> , <code>stroke_width</code> , <code>scale_factor</code>	同上
SurroundingRectangle	<code>mobject</code> , <code>buff</code> , <code>corner_radius</code> , <code>color</code>	同上

Underline	<code>mobject</code> , <code>buff</code> , <code>color</code>	同上
-----------	---	----

常用方法

名称	功能	示例
继承通用操作	<code>scale</code> , <code>move_to</code> , <code>rotate</code> , <code>animate</code> , <code>fade_in/out</code> ...	—

典型陷阱

- 1. 所有匹配器在创建时会复制目标对象的当前边界，随后若原对象移动/缩放，标注不会自动跟随；需手动再 `match_width` / `match_height` 或用 `always_redraw` 。
- 2. `BackgroundRectangle` 默认无边框；设 `stroke_width=0` 则完全无轮廓。
- 3. `Cross` 的 `scale_factor` 是相对目标对角线长度，>1 会超出对象边界。
- 4. `Underline` 始终水平；若对象旋转，下划线不会同步旋转，可用 `SurroundingRectangle` 替代。

6. 图结构

6.1. 无向图 Graph

`Graph` 把“顶点+边”一键变成可视化无向图，支持 10+ 自动布局、顶点/边可换任意 `Mobject`，还能实时增删节点、导入 `NetworkX`。

核心参数

名称	类型	说明	常用示例
<code>vertices</code>	<code>list</code>	顶点列表	<code>[0,1,2]</code>
<code>edges</code>	<code>list[tuple]</code>	边列表	<code>[(0,1),(1,2)]</code>

<code>layout</code>	str	布局方式	<code>"spring"</code> , <code>"tree"</code> , <code>"partite"</code> ...
<code>layout_config</code> / <code>layout_scale</code>	dict / float	微调布局参数	<code>{"k":0.5}</code>
<code>labels</code> / <code>label_fill_color</code>	bool / Color	是否/如何显示顶点标签	<code>labels=True</code>
<code>vertex_type</code> / <code>vertex_config</code>	Mobject / dict	替换顶点形状与样式	<code>vertex_type=Star</code>
<code>edge_type</code> / <code>edge_config</code>	Mobject / dict	替换边形状与样式	<code>edge_type=DashedLine</code>
<code>partitions</code>	list[list]	仅在 <code>layout="partite"</code> 生效	<code>[[0,1],[2,3]]</code>
<code>root_vertex</code>	int/str	仅在 <code>layout="tree"</code> 生效	<code>root_vertex=0</code>

常用布局一览

`spring`, `circular`, `kamada_kawai`, `planar`, `random`, `shell`, `spectral`, `spiral`, `tree`, `partite`

常用方法

名称	功能	示例
<code>add_vertices(v)</code> / <code>add_edges(e)</code>	运行时增节点/边	<code>g.add_edges([(2,3)])</code>
<code>remove_vertices(v)</code> / <code>remove_edges(e)</code>	删除节点/边	<code>g.remove_vertices([3])</code>

<code>change_layout(layout, **kw)</code>	动态切换布局	<code>g.change_layout("circular")</code>
<code>from_networkx(nx_graph)</code>	直接导入 NetworkX 图	<code>Graph.from_networkx(nx_graph)</code>

典型陷阱

1. 顶点/边配置字典的 key 必须对应顶点或边本身，如 `{0:{"color":RED}}`，写错 key 会被静默忽略。
2. `layout="tree"` 必须同时指定 `root_vertex`，否则报错；
`layout="partite"` 必须配 `partitions`。
3. 顶点对象换成 `Text` 等较大 Mobject 时，需调大 `layout_scale` 或手动调 `vertex_config["radius"]`，否则会重叠。
4. 实时增删节点时，布局会重新计算，大量节点 (>100) 建议先静态生成再动画，避免卡顿。

6.2. 有向图 (DiGraph)

`DiGraph` 是 `manim` 库中用于绘制有向图的对象，可表示顶点之间的单向关系（如依赖、流向等），支持动态调整布局、顶点/边样式及拓扑结构。

核心参数

参数	说明	默认值/类型
<code>vertices</code>	顶点列表（如 <code>[0, 1, 2]</code> ）	<code>list</code>
<code>edges</code>	有向边列表（如 <code>[(0,1), (1,2)]</code> ）	<code>list</code>
<code>labels</code>	是否显示顶点标签（ <code>True</code> 或 <code>dict</code> 自定义）	<code>False</code>
<code>label_fill_color</code>	标签背景色	无（透明）
<code>layout</code>	布局方式（ <code>'spring'</code> ， <code>'circular'</code> ， <code>'tree'</code> 等）	<code>'spring'</code>

<code>layout_config</code>	布局微调参数（如 <code>{'k': 0.5}</code> 调整 'spring' 布局的弹性）	<code>dict</code>
<code>vertex_type</code>	顶点图形类型（如 <code>Star</code> , <code>Dot</code> ）	<code>Circle</code>
<code>vertex_config</code>	顶点样式（颜色、大小等，如 <code>{0: {"color": RED}}</code> ）	<code>dict</code>
<code>edge_type</code>	边图形类型（如 <code>Line</code> , <code>Arrow</code> ）	<code>Line</code>
<code>edge_config</code>	边样式（颜色、箭头形状等，如 <code>{(0,1): {"stroke_width": 2}}</code> ）	<code>dict</code>
<code>partitions</code>	层状图的分层顶点列表（需 <code>layout="partite"</code> ）	<code>list</code>
<code>root_vertex</code>	树状图的根节点（需 <code>layout="tree"</code> ）	<code>None</code>

常用方法

方法	功能	示例
<code>add_vertices()</code>	动态添加顶点	<code>graph.add_vertices([3, 4])</code>
<code>remove_vertices()</code>	删除顶点	<code>graph.remove_vertices([0])</code>
<code>add_edges()</code>	添加有向边	<code>graph.add_edges([(2,3)])</code>
<code>remove_edges()</code>	删除边	<code>graph.remove_edges([(0,1)])</code>
<code>change_layout()</code>	切换布局方式	<code>graph.change_layout("circular")</code>
<code>from_networkx()</code>	从 <code>networkx</code> 库导入有向图	<code>DiGraph.from_networkx(nx_graph)</code>

典型陷阱

- 1. 隐藏参数：
 - `edge_config` 中的 `tip_config` （箭头样式）易被忽略，

需显式设置（如 `"tip_shape": ArrowTriangleTip`）。

- `partitions` 和 `root_vertex` 仅在特定布局下生效，未匹配布局时参数无效。

2. 性能瓶颈：

- 动态修改布局（如 `change_layout`）或大规模顶点/边操作可能导致渲染卡顿，建议预计算布局。

3. 版本差异：

- `manim` 不同版本中，`layout` 支持的算法可能变化（如旧版缺少 `'kamada_kawai'`）。
- 树状图的顶点顺序必须从根节点开始，否则渲染异常（与无向图行为不同）。

7. 3D结构

7.1. 三维的点与线

`Dot3D` 是三维空间中的可缩放小球，用来标记关键坐标；

`Line3D` 是任意两点之间的圆柱线段，可画向量或轨迹；

`Arrow3D` 在 `Line3D` 上加圆锥箭头，专门强调方向。

核心参数

对象	参数	类型	说明
Dot3D	<code>point</code>	<code>np.ndarray</code>	球心坐标 (x,y,z)
Dot3D	<code>radius</code>	<code>float</code>	球的半径
Dot3D	<code>color</code>	<code>Color</code>	球体颜色
Dot3D	<code>resolution</code> <code>n</code>	<code>tuple[int,int]</code>	经纬细分，越大越光滑
Line3D	<code>start</code>	<code>np.ndarray</code>	线段起点
Line3D	<code>end</code>	<code>np.ndarray</code>	线段终点
Line3D	<code>thickness</code>	<code>float</code>	圆柱直径
Line3D	<code>color</code>	<code>Color</code>	线段颜色

Arrow3D	<code>start</code>	np.ndarray	箭头起点
Arrow3D	<code>end</code>	np.ndarray	箭头终点
Arrow3D	<code>thickness</code>	float	箭头主体圆柱直径
Arrow3D	<code>height</code>	float	箭头锥体高度
Arrow3D	<code>base_radius</code>	float	箭头锥底半径
Arrow3D	<code>color</code>	Color	箭头整体颜色

常用方法

名称	功能	示例
<code>.move_to(point)</code>	平移到指定坐标	<code>dot.move_to([1,2,3])</code>
<code>.shift(vector)</code>	按向量偏移	<code>line.shift(UP)</code>
<code>.rotate(angle, axis=0UT)</code>	绕指定轴旋转	<code>arrow.rotate(PI/2, axis=Y_AXIS)</code>
<code>.scale(factor)</code>	整体缩放	<code>dot.scale(2)</code>
<code>.set_color(color)</code>	设置颜色	<code>line.set_color(RED)</code>
<code>.animate</code>	创建动画渐变	<code>self.play(dot.animate.shift(RIGHT))</code>
<code>Group(...)</code>	将多个对象编组	<code>Group(dot, line, arrow)</code>
<code>copy()</code>	深度复制生成新对象	<code>new_dot = dot.copy()</code>

Line3D 专属方法

名称	功能	示例
<code>Line3D.parallel_to(line, **kw)</code>	创建与原线段平行的 Line3D	<code>Line3D.parallel_to(line, color=YELLOW)</code>

<code>Line3D.perpendicular_to(line, **kw)</code>	创建与原线段垂直的 Line3D	<code>Line3D.perpendicular_to(line, color=RED)</code>
<code>line.pointify()</code>	返回该线段中心点坐标	<code>mid = line.pointify()</code>

典型陷阱

1. resolution 对 Dot3D 影响渲染性能, > (20,20) 时卡顿显著; 默认即可。
2. thickness 设得过小 (<0.005) 会导致线段在远处完全消失, 需在 camera.frame.reorient 后检查可见性。
3. Arrow3D 的 height/base_radius 与 thickness 比例失衡时箭头会变形;
推荐保持 $height \approx 4 \times base_radius \approx 10 \times thickness$ 。
4. Line3D.parallel_to / perpendicular_to 返回新对象,
旧对象仍保留, 忘记 add() 新线时会出现“线消失”错觉。
5. Manim Community $\leq 0.18.0$ 中 Arrow3D 的 base_radius 参数曾被命名为 tip_radius,
升级后旧代码需替换。

7.2. 立方体 Cube & 棱柱体 Prism

- `Cube` 一键生成标准立方体;
- `Prism` 继承 `Cube`, 可自定义长/宽/高, 快速搭建任意长方体。

核心参数

对象	参数	类型	说明
Cube	<code>side_length</code>	float	立方体边长
Cube	<code>fill_color</code>	Color	表面颜色
Cube	<code>fill_opacity</code>	float	表面透明度
Cube	<code>stroke_width</code>	float	边框线宽

Prism	<code>dimensions</code>	tuple[float]	长、宽、高 (长宽高同值即退化为 Cube)
Prism	<code>fill_color / fill_opacity / stroke_width</code>	同 Cube	同上

常用方法

名称	功能	示例
<code>.move_to(point)</code>	整体平移到指定坐标	<code>cube.move_to([1,2,3])</code>
<code>.shift(vector)</code>	相对平移	<code>prism.shift(LEFT*2)</code>
<code>.rotate(angle, axis=OUT)</code>	绕任意轴旋转	<code>cube.rotate(PI/4, axis=UP)</code>
<code>.scale(factor, **kw)</code>	等比或各向异性缩放	<code>prism.scale([1,1,2])</code>
<code>.set_color(color)</code>	一键换色	<code>cube.set_color(BLUE)</code>
<code>.animate</code>	动画化任意属性	<code>self.play(prism.animate.shift(RIGHT))</code>
<code>VGroup(cube, prism)</code>	编组后整体操作	<code>vg = VGroup(cube, prism)</code>
<code>copy()</code>	深拷贝生成新对象	<code>new_cube = cube.copy()</code>

典型陷阱

- 1. `dimensions` 设成 (0,0,z) 等零宽/零高时模型直接消失，无报错。
- 2. `Cube` 的 `side_length` 与 `Prism` 的 `dimensions` 不能同时传参，混用会触发 `TypeError`。
- 3. `stroke_width=0` 看起来“无边框”，但放大后边缘会出现锯齿；
需改用 `stroke_opacity=0` 完全隐藏。
- 4. `rotate` 默认 `axis=OUT` (Z 轴)，在俯视角场景下旋转效果不明显，
需显式指定 `axis=UP` 或自定义轴。

5. 大量 (>100) `Cube/Prism` 组合动画时, `fill_opacity<1` 会触发透明度排序开销, 帧率骤降;

可预渲染为 `ImageMobject` 或改用 `SurfaceMesh`。

7.3. 圆锥 Cone

`Cone` 是 `Manim` 的“三维锥形生成器”, 可调底面半径、高度、朝向、是否封底, 一键生成实心锥体并支持旋转、指向、阵列动画。

核心参数

名称	类型	默认值	说明
<code>base_radius</code>	float	1.0	圆锥底面半径
<code>height</code>	float	2.0	圆锥高 (底面中心→顶点)
<code>direction</code>	np.ndarray	OUT (Z+)	圆锥轴线方向向量
<code>show_base</code>	bool	True	是否显示底面圆片
<code>v_range</code>	list[float]	[0,1]	纵向扫描范围, 局部截锥用
<code>u_min</code>	float	0	横向扫描起始角, 可切扇形
<code>checkerboard_colors</code>	bool	False	棋盘格着色, 便于区分法向

常用方法

名称	功能	示例
<code>cone.move_to(point)</code>	整体平移到指定坐标	<code>cone.move_to([1,1,1])</code>
<code>cone.shift(vector)</code>	相对平移	<code>cone.shift(RIGHT*2)</code>
<code>cone.rotate(angle, axis=UP)</code>	绕任意轴旋转	<code>cone.rotate(Pi/3, axis=Y_AXIS)</code>
<code>cone.scale(factor)</code>	等比或各向异性缩放	<code>cone.scale([1,1,0.5])</code>

<code>cone.set_color(color)</code>	一键换色	<code>cone.set_color(RED)</code>
<code>cone.animate</code>	任意属性动画	<code>self.play(cone.animate.scale(1.5))</code>
<code>cone.copy()</code>	深拷贝生成新对象	<code>new = cone.copy()</code>
<code>VGroup(*cones)</code>	编组后整体操作	<code>vg = VGroup(*[cone.copy() for _ in range(5)])</code>

Cone 专属方向接口

名称	功能	示例
<code>cone.get_direction()</code>	返回当前轴线方向向量	<code>d = cone.get_direction()</code>
<code>cone.set_direction(vector)</code>	重新设定圆锥指向	<code>cone.set_direction([1,1,0])</code>

典型陷阱

1. `direction` 向量会被自动归一化，但 0 向量会触发 `ValueError`；确保非零。
2. `show_base=False` 时底面圆片被移除，若仍需底部封闭需自行添加 `Disk`。
3. `v_range` 设为 `[a,b]` 且 $0 \leq a < b \leq 1$ 时生成截锥，但 `a>0` 会在尖端留下空洞，需要额外顶点补洞。
4. `checkerboard_colors=True` 时 `set_color` 会被棋盘格覆盖，需先关闭棋盘格再改色。
5. 大量 (>50) `Cone` 的实时 `set_direction` 运算量大，建议预生成方向或用 `rotate` 替代。

7.4. 球体 Sphere

`Sphere` 是 `Manim` 的三维“万能球生成器”，可调球心、半径、经纬分辨率与透明度，用于几何、物理、天体等任何需要球形的场景。

核心参数

名称	类型	默认值	说明
<code>center</code>	<code>Point3D</code> / <code>np.ndarray</code>	<code>ORIGIN</code>	球心坐标 (x,y,z)

<code>radius</code>	float	1.0	球半径
<code>resolution</code>	tuple[int,int]	(20,20)	(u,v) 方向细分, 越大越光滑
<code>u_range</code>	[float,float]	[0,2 π]	横向角度范围, 可做扇形球
<code>v_range</code>	[float,float]	[0, π]	纵向极角范围, 可做半球/球冠
<code>fill_color</code>	Color	WHITE	表面颜色
<code>fill_opacity</code>	float	0.5	表面透明度 (0-1)
<code>stroke_width</code>	float	2	网格线宽

常用方法

名称	功能	示例
<code>.move_to(point)</code>	整体平移到指定坐标	<code>sphere.move_to([2,-1,1])</code>
<code>.shift(vector)</code>	相对平移	<code>sphere.shift(UP*2)</code>
<code>.rotate(angle, axis=0 UT)</code>	绕任意轴旋转	<code>sphere.rotate(PI/3, axis=Y_AXIS)</code>
<code>.scale(factor, **kw)</code>	等比或各向异性缩放	<code>sphere.scale([1,1,0.5])</code>
<code>.set_color(color)</code>	一键换色	<code>sphere.set_color(RED)</code>
<code>.animate</code>	任意属性动画	<code>self.play(sphere.animate.scale(1.5))</code>
<code>.copy()</code>	深拷贝生成新对象	<code>new = sphere.copy()</code>
<code>VGroup(*spheres)</code>	编组后整体操作	<code>grp = VGroup(*[s.copy() for s in spheres])</code>

典型陷阱

1. `resolution` 每增加一倍, 顶点数 $\approx \times 4$, $>128 \times 128$ 时实时渲染帧率骤降;

高分辨率场景建议离线渲染或改用纹理球。

2. `u_range / v_range` 若未完整覆盖 $[0, 2\pi]/[0, \pi]$ 会得到开口的“残球”，需额外添加平面封口才能保持体积感。
3. `fill_opacity < 1` 时，大量透明 Sphere (> 50) 会触发深度排序开销；可改用 shader-based 点精灵或 ImageMobject 替代。
4. `sphere.set_color` 仅改变表面着色，若需贴图需改用 `TexturedSurface`；直接给 `Sphere` 贴图会被忽略。
5. Manim CE ≤ 0.18 中 `resolution` 只接受 int，新版改为 tuple，旧代码需统一升级。

7.5. 圆环面 Torus

`Torus` 是 `Manim` 的“甜甜圈生成器”，通过 `major/minor` 半径控制环体大小与粗细，配合 `u/v_range` 可切片、截断或扁平化，一键生成可旋转、可扭曲的拓扑圆环。

核心参数

名称	类型	默认值	说明
<code>major_radius</code>	float	3.0	环体中心到管道中心的距离 (轮胎外径)
<code>minor_radius</code>	float	1.0	管道自身半径 (轮胎厚度)
<code>resolution</code>	tuple[int,int]	(24,24)	(u,v) 方向细分，越大越光滑
<code>u_range</code>	tuple[float,float]	$[0, 2\pi]$	横向角度范围，可切出部分环
<code>v_range</code>	tuple[float,float]	$[0, 2\pi]$	纵向角度范围，可压成扁平环
<code>fill_color</code>	Color	WHITE	表面颜色
<code>fill_opacity</code>	float	0.8	表面透明度

<code>stroke_width</code>	float	1	网格线宽
---------------------------	-------	---	------

常用方法

名称	功能	示例
<code>.move_to(point)</code>	整体平移到指定坐标	<code>torus.move_to([0,0,0])</code>
<code>.shift(vector)</code>	相对平移	<code>torus.shift(UP*2)</code>
<code>.rotate(angle, axis=UP)</code>	绕任意轴旋转	<code>torus.rotate(PI/3, axis=X_AXIS)</code>
<code>.scale(factor)</code>	等比或各向异性缩放	<code>torus.scale([1,1,0.5])</code>
<code>.set_color(color)</code>	一键换色	<code>torus.set_color(RED)</code>
<code>.animate</code>	任意属性动画	<code>self.play(torus.animate.rotate(PI))</code>
<code>.copy()</code>	深拷贝生成新对象	<code>new = torus.copy()</code>
<code>VGroup(*torii)</code>	编组后整体操作	<code>grp = VGroup(*[t.copy() for t in torii])</code>

典型陷阱

1. `major_radius ≤ minor_radius` 时环体自交，渲染结果呈“麻花”状，应确保 `major > minor`。
2. `resolution` 每提高一倍，顶点数 $\approx \times 4$ ， $>100 \times 100$ 时实时交互帧率骤降；
建议离线渲染或改用低模+法线贴图。
3. `u_range/v_range` 设为非完整区间 `[a,b]` 时会在缺口处留下“裂缝”，
若需封闭须手动补面或改用 `SurfaceMesh`。
4. `v_range=(0,π)` 会生成“半环面”，但底部法线朝下，光照可能异常；
需加 `.flip_faces()` 或调整光源。
5. 旧版 `Manim CE ≤0.18` 中 `v_range` 默认 $[0,\pi]$ （半球），
新版已统一为 `[0,2π]`，升级后注意检查外观是否突变。

7.6. 圆柱体 Cylinder

`Cylinder` 是 `Manim` 的“可定向圆柱生成器”，可调半径、高度、方向、是否封顶，支持动态加底、旋转、缩放，用于几何演示、物理模拟和工程设计。

核心参数

名称	类型	默认值	说明
<code>radius</code>	float	1.0	圆柱底面半径
<code>height</code>	float	2.0	圆柱高（沿主轴长度）
<code>direction</code>	np.ndarray	OUT (Z+)	圆柱主轴方向向量
<code>show_ends</code>	bool	True	是否显示上下底面圆片
<code>v_range</code>	tuple[float,float]	[0,1]	纵向参数化范围，可切出“管段”
<code>resolution</code>	tuple[int,int]	(24,24)	(u,v) 网格细分，越大越光滑
<code>fill_color</code>	Color	WHITE	表面颜色
<code>fill_opacity</code>	float	0.8	表面透明度
<code>stroke_width</code>	float	1	边框线宽

常用方法

名称	功能	示例
<code>.move_to(point)</code>	整体平移到指定坐标	<code>cyl.move_to([2,0,0])</code>
<code>.shift(vector)</code>	相对平移	<code>cyl.shift(LEFT*2)</code>
<code>.rotate(angle, axis=UP)</code>	绕任意轴旋转	<code>cyl.rotate(PI/2, axis=RIGHT)</code>
<code>.scale(factor)</code>	等比或各向异性缩放	<code>cyl.scale([1,1,0.5])</code>
<code>.set_color(color)</code>	一键换色	<code>cyl.set_color(RED)</code>

<code>.animate</code>	任意属性动画	<code>self.play(cyl.animate.shift(UP))</code>
<code>.copy()</code>	深拷贝生成新对象	<code>new = cyl.copy()</code>
<code>VGroup(*cylinders)</code>	编组后整体操作	<code>grp = VGroup(cyl, cyl2)</code>

Cylinder 专属方法

名称	功能	示例
<code>cylinder.add_bases()</code>	在运行时动态补上下底面	<code>cyl.add_bases()</code>
<code>cylinder.get_direction()</code>	返回当前主轴方向向量	<code>d = cyl.get_direction()</code>
<code>cylinder.set_direction(vec)</code>	重新设定圆柱朝向	<code>cyl.set_direction([1,1,0])</code>

典型陷阱

1. `direction` 会被自动归一化，传入零向量触发 `ValueError`；需确保非零。
2. `show_ends=False` 时无底面，若之后调用 `add_bases()` 会一次性补两个圆片，无法单独加顶或底。
3. `v_range` 设为 `[a,b]` 且 `0<a<b<1` 时生成“管段”，缺口处无端面，需要额外补片才能封闭。
4. `resolution` 每提高一倍，顶点数 $\approx \times 4$ ， $>100 \times 100$ 时实时帧率骤降；建议离线渲染或改用纹理贴图。
5. Manim CE ≤ 0.18 中 `set_direction` 旧实现存在 180° 翻转抖动， ≥ 0.19 已修复，升级后注意测试旋转动画。

7.7. 曲面 Surface

`Surface` 是 `Manim` 的“参数方程绘图器”，只需给出 $u \rightarrow v \rightarrow (x,y,z)$ 的函数即可生成任意三维曲面，并支持棋盘格或数值渐变着色，用于数学、物理、工程可视化。

核心参数

名称	类型	默认值	说明
<code>func</code>	<code>Callable[[u,v], (x,y,z)]</code>	必填	参数方程函数，输入 u,v 返回三维点
<code>u_range</code>	<code>tuple[float,float]</code>	<code>[-1,1]</code>	u 向定义域
<code>v_range</code>	<code>tuple[float,float]</code>	<code>[-1,1]</code>	v 向定义域
<code>resolution</code>	<code>tuple[int,int]</code>	<code>(24,24)</code>	(u,v) 细分网格，越大越光滑
<code>stroke_color</code>	<code>Color</code>	<code>WHITE</code>	网格线颜色
<code>stroke_width</code>	<code>float</code>	<code>1</code>	网格线宽
<code>fill_color</code>	<code>Color</code>	<code>WHITE</code>	表面颜色
<code>fill_opacity</code>	<code>float</code>	<code>0.8</code>	表面透明度

常用方法

名称	功能	示例
<code>.move_to(point) / .shift(vector)</code>	整体平移	<code>surf.shift(UP*2)</code>
<code>.rotate(angle, axis=UP)</code>	绕任意轴旋转	<code>surf.rotate(PI/2, axis=X_AXIS)</code>
<code>.scale(factor)</code>	等比或各向异性缩放	<code>surf.scale([1,1,0.5])</code>
<code>.set_color(color)</code>	一键换色	<code>surf.set_color(BLUE)</code>
<code>.animate</code>	任意属性动画	<code>self.play(surf.animate.rotate(PI))</code>
<code>.copy()</code>	深拷贝生成新对象	<code>new = surf.copy()</code>
<code>VGroup(*surfs)</code>	编组后整体操作	<code>grp = VGroup(s1,s2)</code>

Surface 专属着色方法

名称	功能	示例
<code>set_fill_by_checkerboard(c1,c2)</code>	棋盘格双色填充	<code>surf.set_fill_by_checkerboard(RED,BLUE)</code>

<code>set_fill_by_value(axes, colorscale, axis)</code>	按函数值渐变着色	<code>surf.set_fill_by_value(axes, [(RED,-1),(GREEN,1)], axis=2)</code>
--	----------	---

典型陷阱

1. `func` 必须返回三元组 (x,y,z) 而非仅 z，否则直接报错 `TypeError`。
2. `resolution` 每提高一倍，顶点数 $\approx \times 4$ ，>128×128 时实时帧率骤降；高分辨率场景建议离线渲染。
3. `u_range/v_range` 设为子区间会在曲面边缘留下“裂缝”，需额外补片或改用 `ParametricSurface` 并手动封口。
4. `set_fill_by_value` 的 `colorscale` 需严格升序排列值域，否则颜色映射会反向或报错。
5. 旧版 Manim CE ≤ 0.18 中 `Surface` 默认 `fill_opacity=0`，导致创建后“看不见”；新版已默认 0.8，升级后注意测试透明度。

7.8. 多面体 Polyhedron & 正多面体

`Polyhedron` 是“万能多面体工厂”，顶点和面表驱动，可拼出任意凸/凹体；

`Tetrahedron`、`Octahedron`、`Dodecahedron`、`Icosahedron` 则是开箱即用的正多面体，边长一设即可生成。

核心参数

对象	参数	类型	说明
通用正多面体	<code>edge_length</code>	float	统一边长
<code>Polyhedron</code>	<code>vertex_coords</code>	list[list[float]]	顶点坐标列表
<code>Polyhedron</code>	<code>faces_list</code>	list[list[int]]	面的顶点索引列表
<code>Polyhedron</code>	<code>faces_config</code>	dict	面级样式（颜色、透明度...）
<code>Polyhedron</code>	<code>graph_config</code>	dict	图结构配置（边权重等）

共享	<code>fill_color</code>	Color	整体颜色
共享	<code>fill_opacity</code>	float	整体透明度
共享	<code>stroke_width</code>	float	边线宽度

常用方法

名称	功能	示例
<code>.move_to(point) / .shift(vec)</code>	整体平移	<code>oct.move_to([2,0,0])</code>
<code>.rotate(angle, axis=UP)</code>	绕任意轴旋转	<code>dode.rotate(PI/4, axis=OUT)</code>
<code>.scale(factor)</code>	等比或各向异性缩放	<code>tet.scale(2)</code>
<code>.faces.set_color(color)</code>	一键统一换色	<code>ico.faces.set_color(RED)</code>
<code>.faces[i].set_color(c)</code>	单面着色	<code>p.faces[0].set_color(GREEN)</code>
<code>.faces.set_opacity(o)</code>	统一透明度	<code>tet.faces.set_opacity(0.5)</code>
<code>.animate</code>	任意属性动画	<code>self.play(ico.animate.rotate(PI))</code>
<code>.copy()</code>	深拷贝生成新对象	<code>new = cube.copy()</code>
<code>VGroup(*polyhedra)</code>	编组整体操作	<code>grp = VGroup(o,t)</code>

典型陷阱

- 1. `Polyhedron` 的 `faces_list` 索引必须全部落在 `vertex_coords` 范围内，越界即报 `IndexError`；
复制外部模型时易漏顶点。
- 2. `edge_length` 只在正多面体类生效，`Polyhedron` 自定义时无效；想放大需用 `.scale()`。
- 3. `faces_config` 字典键必须为面的索引 `int`，写成字符串会被静默忽略。
- 4. 大量 (>100) 正多面体同时旋转时，`fill_opacity<1` 会触发透明度排序瓶颈；
可先预渲染成 Mesh 或改用实体着色。

5. Manim CE ≤ 0.18 中 `Dodecahedron` 的默认边长为 1.0 但顶点坐标缩放因子与新版不同，升级后同一场景可能出现大小差异，需重新统一 `scale`。

8. 坐标系

8.1. 数轴 NumberLine

`NumberLine` 是 `Manim` 的“一维坐标轴生成器”，可调范围、长度、刻度、箭头、数值标签及比例尺，用来动态展示实数、坐标或函数映射。

核心参数

名称	类型	默认值	说明
<code>x_range</code>	<code>Sequence[float]</code>	<code>[-5,5,1]</code>	起止值与步长
<code>length</code>	<code>float</code>	6	轴线可视长度 (单位: Manim 空间)
<code>unit_size</code>	<code>float</code>	<code>length/(max-min)</code>	每单位长度对应的屏幕距离
<code>include_ticks</code>	<code>bool</code>	<code>True</code>	是否显示刻度线
<code>tick_size</code>	<code>float</code>	0.1	刻度线长度
<code>numbers_with_elongated_ticks</code>	<code>Iterable[float]</code>	<code>[]</code>	需要加粗/加长的特殊刻度
<code>longer_tick_multiple</code>	<code>int</code>	2	特殊刻度线加长的倍数
<code>rotation</code>	<code>float</code>	0	整体旋转角度 (弧度)
<code>stroke_width</code>	<code>float</code>	2	轴线粗细
<code>include_tip</code>	<code>bool</code>	<code>False</code>	是否带箭头
<code>tip_width / tip_height</code>	<code>float</code>	0.15	箭头宽高
<code>tip_shape</code>	<code>ArrowTip</code>	<code>ArrowTip</code>	箭头样式

<code>include_numbers</code>	bool	True	是否显示数字标签
<code>scaling</code>	<code>_ScaleBase</code>	LinearBase	比例尺： 线性、对数、自定义
<code>font_size</code>	float	24	数字标签字号
<code>label_direction</code>	Sequence[float]	DOWN	数字相对轴方向
<code>label_constructor</code>	VObject	MathTex	数字对象类型
<code>line_to_number_buff</code>	float	0.1	数字与轴线间距
<code>decimal_number_config</code>	dict	{}	数字样式 (颜色、小数位...)
<code>numbers_to_exclude</code> / <code>numbers_to_include</code>	Iterable[float]	[]	黑名单/白名单数字

常用方法

名称	功能	示例
<code>.add_labels(dict)</code>	指定位置添加文字标签	<code>nl.add_labels({3: "a"})</code>
<code>.add_numbers()</code> / <code>.add_ticks()</code>	手动补数字或刻度	<code>nl.add_numbers([0, π, 2π])</code>
<code>.get_labels()</code> / <code>.get_number_mobject()</code>	获取标签对象	<code>lbls = nl.get_labels()</code>
<code>.n2p(value)</code>	数值→屏幕坐标	<code>pos = nl.n2p(3.5)</code>
<code>.p2n(point)</code>	屏幕坐标→数值	<code>val = nl.p2n(dot.get_center())</code>
<code>.animate</code>	任意属性动画	<code>self.play(nl.animate.shift(LEFT))</code>

典型陷阱

- 1. 当 `scaling` 为 `LogBase` 且 `x_range` 含 ≤ 0 值时直接报错；需保证区间 >0 。
- 2. 修改 `x_range` 后必须重新 `.add_numbers()`，否则旧数字对象残留。
- 3. `.n2p()` 返回三维坐标，若将结果直接给 2D Mobject 需取前两维（`pos[:2]`）。
- 4. 大量 (>50) 数轴同时存在时，`include_numbers=True` 会导致字体缓存暴增；可改用批量 `TeX` 标签。
- 5. Manim CE ≤ 0.18 中 `tip_shape` 参数无效，始终为默认箭头； ≥ 0.19 支持自定义箭头。

8.2. 直角平面 NumberPlane

`NumberPlane` 是 `Manim` 的“二维坐标系画布”，自带网格、轴与比例尺，可一键绘制函数曲线、几何图形，并支持自定义网格样式、轴比例和坐标映射。

核心参数

名称	类型	默认值	说明
<code>x_range</code>	<code>Sequence[float]</code>	<code>[-8,8,1]</code>	横轴范围与刻度间隔
<code>y_range</code>	<code>Sequence[float]</code>	<code>[-4,4,1]</code>	纵轴范围与刻度间隔
<code>x_length</code>	<code>float</code>	<code>8</code>	画布可视宽度
<code>y_length</code>	<code>float</code>	<code>4</code>	画布可视高度
<code>background_line_style</code>	<code>dict</code>	<code>{"stroke_color": BLUE, ...}</code>	主网格线样式
<code>faded_line_style</code>	<code>dict</code>	<code>{"stroke_color": GREY, ...}</code>	淡化网格线样式
<code>faded_line_ratio</code>	<code>int</code>	<code>1</code>	淡化线与主线间隔倍数
<code>make_smooth_after_applying_functions</code>	<code>bool</code>	<code>False</code>	函数绘制后是否平滑曲线

常用方法

名称	功能	示例
<code>.plot_parametric_curve(func, t_range)</code>	绘制参数曲线	<code>plane.plot_parametric_curve(lambda t: [t, np.sin(t)])</code>
<code>.plot_implicit_curve(f, ...)</code>	绘制隐函数曲线	<code>plane.plot_implicit_curve(lambda x,y: x²+y²-1)</code>
<code>.coords_to_point(x, y)</code>	坐标 → 屏幕位置	<code>dot.move_to(plane.coords_to_point(2,3))</code>
<code>.point_to_coords(point)</code>	屏幕位置 → 坐标	<code>x,y = plane.point_to_coords(dot.get_center())</code>
<code>.get_axes()</code>	获取 (x_axis, y_axis)	<code>x_axis, y_axis = plane.get_axes()</code>
<code>.animate</code>	整体动画	<code>self.play(plane.animate.shift(LEFT))</code>
<code>.copy()</code>	深拷贝	<code>new_plane = plane.copy()</code>

典型陷阱

- 坐标与屏幕尺寸不匹配：`x_range` 与 `x_length`、`y_range` 与 `y_length` 独立设置，比例差异大时会出现“网格被压扁/拉长”的错觉，需手动统一比例。
- `faded_line_ratio=0` 会关闭淡化线，但保留字典样式仍可能渲染空对象，建议直接省略 `faded_line_style`。
- 大量 (>50) 曲线绘制时，`make_smooth_after_applying_functions=True` 会显著增加顶点数，导致帧率下降；可改用 `LineGraph` 或离线渲染。
- `plot_parametric_curve` 默认 `t_range` 步长 0.01，若区间过大易生成百万顶点；应根据函数复杂度手动调大步长。
- Manim CE ≤0.18 中 `NumberPlane` 默认无箭头，≥0.19 可通过 `x_axis_config/y_axis_config` 添加箭头，升级后注意样式差异。

8.3. 复数平面 ComplexPlane

`ComplexPlane` 是 `NumberPlane` 的“复数特化版”，横轴为实部、纵轴为虚部，提供 `n2p/p2n` 快速映射与复数运算动画，一键展示复数几何。

核心参数（与 `NumberPlane` 相同）

名称	类型	默认值	说明
<code>x_range</code>	<code>Sequence[float]</code>	<code>[-8,8,1]</code>	实轴范围与刻度间隔
<code>y_range</code>	<code>Sequence[float]</code>	<code>[-4,4,1]</code>	虚轴范围与刻度间隔
<code>x_length / y_length</code>	<code>float</code>	<code>8 / 4</code>	画布可视宽/高
<code>background_line_style / faded_line_style</code>	<code>dict</code>	同 <code>NumberPlane</code>	主/淡化网格样式
<code>faded_line_ratio</code>	<code>int</code>	<code>1</code>	淡化线倍数

常用方法（继承自 `NumberPlane`）

名称	功能	示例
<code>.add_coordinates()</code>	一键添加实/虚轴刻度	<code>plane.add_coordinates()</code>
<code>.n2p(z)</code>	复数 → 屏幕坐标	<code>dot.move_to(plane.n2p(3+2j))</code>
<code>.p2n(point)</code>	屏幕坐标 → 复数	<code>z = plane.p2n(dot.get_center())</code>
<code>.plot_complex_function(f, **kw)</code>	绘制复数映射曲线	<code>plane.plot_complex_function(lambda z: z**2)</code>
<code>.move_to / .shift / .rotate / .scale</code>	同 <code>NumberPlane</code>	<code>plane.rotate(PI/4)</code>

<code>.animate</code>	整体动画	<code>self.play(plane.animate.scale(1.2))</code>
-----------------------	------	--

典型陷阱

1. 复数平面默认原点与屏幕原点重合；若需平移整个平面请用 `plane.shift()`，而非修改 `x_range/y_range`（后者会重算网格）。
2. `n2p` 输出 3D 坐标，2D 场景需取前二维：`dot.move_to(plane.n2p(z)[:2])`。
3. `plot_complex_function` 默认步长 0.01，大区间易生成百万级顶点；需手动调大步长或改用采样点连线。
4. 复数含 `NaN/Inf` 时 `n2p` 直接报错；需在映射前过滤非法值。
5. Manim CE ≤ 0.18 中 `ComplexPlane` 类缺失，需手动 `from manim.mobject.complex_plane import ComplexPlane`； ≥ 0.19 已内置。

8.4. 极坐标平面 PolarPlane

`PolarPlane` 是 `Manim` 的“极坐标画布”，用角度 θ 与半径 r 定位点，自带同心圆与径向网格，可一键绘制极函数、玫瑰线、心形线等图案。

核心参数

名称	类型	默认值	说明
<code>radius_max</code>	float	4.0	最大极径
<code>radius_step</code>	float	1.0	同心圆刻度间隔
<code>azimuth_units</code>	str	"PI radians"	角度单位： PI/TAU/degrees/gradians/None
<code>azimuth_step</code>	float	$\pi/4$	角度刻度步长
<code>azimuth_direction</code>	str	"CCW"	角度递增方向：CCW 逆时针 / CW 顺时针

<code>azimuth_offset</code>	float	0	0° 起始偏移
<code>azimuth_label_font_size</code>	float	24	角度标签字号
<code>azimuth_label_buff</code>	float	0.15	角度标签离轴线距离
<code>size</code>	float	None	画布直径；不填时自动 = $\text{radius_max} \times 2$
<code>background_line_style</code> / <code>faded_line_style</code>	dict	同 NumberPlane	主/淡化网格线样式
<code>faded_line_ratio</code>	int	1	淡化线倍率

常用方法

名称	功能	示例
<code>.add_coordinates()</code>	添加同心圆与角度标签	<code>plane.add_coordinates()</code>
<code>.plot_polar_graph(r_func, θ_range, **kw)</code>	绘制极函数图像	<code>plane.plot_polar_graph(lambda θ:2np.sin(3θ), [0,2PI], color=RED)</code>
<code>.coords_to_point(r,θ)</code>	极坐标 → 屏幕坐标	<code>dot.move_to(plane.coords_to_point(2,PI/3))</code>
<code>.point_to_coords(point)</code>	屏幕坐标 → 极坐标	<code>r,θ = plane.point_to_coords(dot.get_center())</code>
<code>.rotate(angle, axis=OUT)</code>	整体旋转网格	<code>plane.rotate(PI/4)</code>
<code>.animate</code>	动画化任意属性	<code>self.play(plane.animate.scale(1.2))</code>
<code>.copy()</code>	深拷贝	<code>new_plane = plane.copy()</code>

典型陷阱

- 1. `azimuth_units` 设为 `None` 时角度标签为 0–1 小数，易误解；需手动格式化。
- 2. `azimuth_step` 过小 ($<\pi/8$) 且 `radius_max` 大时，角度标签重叠；需调大字号或减小步长。
- 3. `plot_polar_graph` 默认 θ 步长 0.01，高阶玫瑰线会生成百万顶点；建议显式增加步长。
- 4. 顺时针方向 CW 时，角度标签仍按 CCW 书写，视觉效果相反；
需同时修改 `azimuth_offset` 调整 0° 位置。
- 5. Manim CE ≤ 0.18 没有 `PolarPlane` 类，需手动从 `manim.mobject.coordinate_systems` 导入；
 ≥ 0.19 已内置。

8.5. 通用二维坐标系 Axes

`Axes` 是 `Manim` 的“万能二维坐标系”，可独立配置 x、y 轴范围、长度、颜色、箭头、刻度与比例尺，并提供 `coords`↔`screen` 映射及一键绘图接口，是函数曲线、数据可视化的基础容器。

核心参数

名称	类型	默认值	说明
<code>x_range</code>	<code>Sequence[float]</code>	<code>[-5,5,1]</code>	横轴范围 [min,max,step]
<code>y_range</code>	<code>Sequence[float]</code>	<code>[-3,3,1]</code>	纵轴范围 [min,max,step]
<code>x_length</code> / <code>y_length</code>	<code>float</code>	<code>6 / 4</code>	轴在屏幕上的可视长度
<code>axis_config</code>	<code>dict</code>	<code>{}</code>	全局轴通用样式
<code>x_axis_config</code> / <code>y_axis_config</code>	<code>dict</code>	<code>{}</code>	单轴专属样式 (颜色、箭头、刻度等)
<code>tips</code>	<code>bool</code>	<code>True</code>	是否自动在正方向加箭头

`axis_config` / `x_axis_config` / `y_axis_config` 常用子键

键	类型	示例	作用
<code>stroke_color</code>	Color	RED	轴线颜色
<code>stroke_width</code>	float	2	轴线粗细
<code>include_numbers</code>	bool	True	是否显示数字
<code>include_tip</code>	bool	True	是否显示箭头
<code>tip_shape</code>	ArrowTip	StealthTip	箭头形状
<code>tip_length</code>	float	0.25	箭头长度
<code>numbers_with_elongated_ticks</code>	list[float]	[0,π]	加长刻度列表
<code>font_size</code>	float	20	数字字号
<code>scaling</code>	_ScaleBase	LogBase(10)	线性/对数/自定义比例尺

常用方法

名称	功能	示例
<code>.plot(func, x_range=[a,b], **kw)</code>	绘制 $y=f(x)$ 曲线	<code>axes.plot(lambda x: x**2, [-2,2])</code>
<code>.plot_parametric_curve(func, t_range)</code>	绘制参数曲线	<code>axes.plot_parametric_curve(lambda t: [t,np.sin(t)])</code>
<code>.plot_implicit_curve(f)</code>	绘制隐函数	<code>axes.plot_implicit_curve(lambda x,y: x²+y²-1)</code>
<code>.coords_to_point(x,y)</code>	坐标 → 屏幕位置	<code>dot.move_to(axes.coords_to_point(1,2))</code>
<code>.point_to_coords(point)</code>	屏幕位置 → 坐标	<code>x,y = axes.point_to_coords(dot.get_center())</code>

<code>.get_axes()</code>	返回 (x_axis, y_axis)	<code>xa, ya = axes.get_axes()</code>
<code>.get_x_axis_label(label, **kw)</code>	添加 x 轴标签	<code>axes.get_x_axis_label(MathTex("x"))</code>
<code>.animate</code>	整体动画	<code>self.play(axes.animate.rotate($\pi/6$))</code>

典型陷阱

1. `tips=True` 时箭头会额外占用轴长，导致可视刻度范围略小于 `x_range` ；
若需精确范围请手动设 `tips=False` 并额外加 `Arrow` 。
2. `x_range` 步长设为 0 或负数会触发 `ValueError` ；动态范围请用 `DecimalNumber` 或更新器。
3. `plot` 默认步长 0.01，大区间函数会产生百万级顶点；建议按曲率手动增大步长或使用 `LineGraph` 。
4. 对数坐标轴 `LogBase` 时，`x_range` 需全部 >0，否则直接报错。
5. Manim CE ≤0.18 中 `y_axis_config` 的 `include_tip` 无效，需升级 ≥0.19 才能双轴同时加箭头。

8.6. 通用三维坐标系 ThreeDAxes

`ThreeDAxes` 是 `Manim` 的“三维直角坐标系工厂”，可独立设定 `X/Y/Z` 轴的范围、长度、颜色与箭头，并自带 `plot_surface`、`coords_to_point` 等接口，用于 3D 函数、曲面与数据可视化。

核心参数

名称	类型	默认值	说明
<code>x_range / y_range / z_range</code>	<code>Sequence[float]</code>	<code>[-5,5,1]</code>	各轴范围 [min,max,step]

<code>x_length / y_length / z_length</code>	float	6 / 4 / 3	轴在屏幕上的可视长度
<code>x_axis_config / y_axis_config / z_axis_config</code>	dict	{}	单轴专属样式 (颜色、箭头、刻度等)
<code>axis_config</code>	dict	{}	全局轴通用样式
<code>z_normal</code>	np.ndarray	OUT	Z 轴“正向”方向（可翻转坐标系）
<code>light_source</code>	Sequence[float]	[-10,10,10]	默认光源位置，影响曲面阴影
<code>tips</code>	bool	True	是否在各轴正方向自动加箭头

`axis_config / *_axis_config` 常用子键

键	示例	作用
<code>stroke_color</code>	RED	轴线颜色
<code>stroke_width</code>	2	轴线粗细
<code>include_tip</code>	True	是否显示箭头
<code>tip_shape</code>	StealthTip	箭头样式
<code>include_numbers</code>	True	显示刻度数字
<code>font_size</code>	20	数字字号
<code>scaling</code>	LogBase(10)	对数/线性比例

常用方法

名称	功能	示例
<code>.plot_surface(f, u_range, v_range, **kw)</code>	绘制 $z=f(u,v)$ 曲面	<code>axes.plot_surface(lambda u,v: u*v, [-2,2], [-1,1])</code>

<code>.plot_parametric_surface</code> (f, **kw)	绘制参数曲面	<code>axes.plot_parametric_surface(lambda u,v: [u,v,u²+v²])</code>
<code>.coords_to_point(x,y,z)</code>	坐标 → 屏幕位置	<code>dot.move_to(axes.coords_to_point(1,2,3))</code>
<code>.point_to_coords(point)</code>	屏幕位置 → 坐标	<code>x,y,z = axes.point_to_coords(dot.get_center())</code>
<code>.get_axis_labels(x_tex, y_tex, z_tex)</code>	一次性返回三轴标签	<code>labels = axes.get_axis_labels("x","y","z")</code>
<code>.add_coordinates()</code>	添加网格刻度数字	<code>axes.add_coordinates()</code>
<code>.animate</code>	整体动画	<code>self.play(axes.animate.rotate(PI/4, axis=UP))</code>

典型陷阱

1. 设置 `z_range` 含有 0 且使用 `LogBase` 会报错；需保证区间 >0 或改用 `LinearBase`。
2. `plot_surface` 默认 `u/v` 步长 0.1，大区间易生成百万级顶点；
应手动调大步长或降低 `resolution`。
3. `coords_to_point` 返回 3D 坐标，2D 场景下需投影或取前两维使用。
4. `light_source` 改变后需重新渲染才能看到阴影变化；实时预览不会立即更新。
5. Manim CE ≤ 0.18 中 `z_axis_config` 的 `include_tip` 无效，需升级 ≥ 0.19 才能在 Z 轴显示箭头。